Harbor Watch: rotating camera feed in the UI

Action plan for showing the 360° rotating camera in the existing UI, and finding why the feed is slow.

Goal

- Show the rotating camera feed in the same React UI.
- Use the same backend and the same Java jar. Nothing changes on the jar or UI side.
- Find out whether the slow feed comes from the front/back panorama stitching or from the UI.

Current setup

- Chain: hw_v3.py (python model backend) to Java jar to React UI (`npm run dev`).
- hw_v3.py emits `frame` on Socket.IO namespaces `/front` and `/rear`, port 12010.
- nvdec.py is the GPU decode file: PyAV reads packets, NVDEC decodes, frames stay on the GPU as RGB tensors.
- Stitching is `split_concat.run`: a 3840x2160 frame becomes a 7680x1080 panorama.

Likely causes of delay in hw_v3.py

1. CPU decode of 4K plus `cv2.resize` on the CPU before the GPU copy.
2. No new-frame check, so the same frame is processed again and again.
3. The GPU loop runs inside eventlet with `monkey_patch`, so blocking CUDA and JPEG work stalls websocket sends.
4. JPEG encode starts without waiting for the D2H copy on `stream_d2h`.
5. `run_detection = True` and `run_classification = True` are hardcoded, so full detection and classification always run.
6. Base64 JPEG over Socket.IO, and the UI possibly queueing frames.

Steps

1. Run nvdec.py alone. Check the `[cam][stats]` line: decode fps should match the camera fps, and decode and copy ms should be small.
2. Run `python nvdec_ui.py` (stitch on). Read the `[server]` line printed every 5 s.
3. Run `python nvdec_ui.py --no-stitch`. Compare the stitch time and the overall lag with step 2.
4. Start the server with `HW_DEBUG=1`, then open `ui_latency_test.html` in a browser. Note received fps, shown fps, image decode time and server-to-browser age.
5. Open the real React UI and compare it with the test page.
6. Decide using the table below.

Reading the results

What you see	Meaning	Next step
Decode fps below camera fps	Network or decode problem	Check RTSP transport (udp/tcp), packet loss, camera load
stitch ms is large, and drops with --no-stitch	Stitching is the cost	Optimise or move the stitch, or run it at lower rate
Server numbers small, test page smooth, React UI laggy	UI problem	See UI fixes
Server numbers small, test page also laggy	Transport or payload	Lower JPEG quality or size, move to binary instead of base64
Age grows over time	Frames are queueing somewhere	Keep latest-frame only, drop old frames

Rule of thumb: a few tens of ms of server time per frame is fine. If the age keeps climbing, something is queueing.

UI fixes (if the UI is the cause)

- Keep only the newest frame and drop the rest.
- Avoid `setState` on every frame. Draw to a canvas or set `img.src` directly.
- Decode off the main thread (`img.decode()` or `createImageBitmap`).
- Send binary JPEG instead of base64 if the jar and UI allow it.

Integrating into hw_v3.py (if "same backend" means one process)

1. Replace hw_v3's old `RTSPReader` with the nvdec.py reader.
2. Use `wait_new` so only new frames are processed.
3. Sync the D2H copy before JPEG encode, and encode as RGB (nvdec gives RGB, the old reader gave BGR).
4. Remove the hardcoded `run_detection = True` and `run_classification = True`.
5. Move JPEG encode and emit out of the GPU loop.

Open questions

- Does the rotating camera send the same 3840x2160 stacked layout as the front camera? `split_concat` only works on that. If not, use `--no-stitch`.
- Should the rotating camera use /front, /rear, or its own namespace in the UI?
- Does the Java jar only relay the payload, or does it validate fields?
- Should this run as a second process, or inside hw_v3.py?

Files

- `nvdec_ui.py`: NVDEC feed to Socket.IO, same payload as hw_v3.py, with stage timing logs.
- `ui_latency_test.html`: bare browser page that measures UI-side lag.
- `nvdec.py`: existing GPU reader and OpenGL viewer.